11

Hooks por Agent: o contrato virando código

System prompt é pedido; hook é lei. Um agent pode "decidir" ignorar uma instrução do prompt — mas não decide nada contra um script que bloqueia a ação com exit 2. Este material mostra como dar a cada agent os guardrails do seu domínio: o reviewer que não escreve, o backend que só roda testes do seu módulo, o QA que não toca em mais nada.

A IDEIA CENTRAL DESTE MATERIAL

Hooks globais protegem o projeto; hooks por agent protegem o contrato de cada agent. A composição correta usa os dois — e no Claude Code há um motivo técnico que torna os hooks por agent obrigatórios, não opcionais.

1 Dois níveis, duas responsabilidades

NÍVEL	ONDE VIVE	QUE REGRA CARREGA
Hooks globais	`settings.json`	O que vale para qualquer agent: nunca ler .env, formatar após edição, nunca commitar direto na main. (Material 12.)
Hooks por agent	Frontmatter do agent	Regras do domínio: reviewer não escreve, backend só roda testes, parser valida o output antes de retornar.

> **Detalhe que muda tudo (Claude Code):** hooks do `settings.json` não disparam automaticamente para tool calls feitas *dentro* de sub-agents — só para a sessão principal. Ou seja: hooks no frontmatter não são boa prática, são **necessidade** para guardrails funcionarem quando você delega. (Comportamento da versão à época — teste na sua.)

2 Como declarar no Claude Code

Na seção `hooks` do frontmatter em `.claude/agents/nome.md`:

```
---
name: nome-do-agent
description: ...
tools: ...
hooks:
  PreToolUse:                    # antes da tool executar
    - matcher: "Bash"
      hooks:
        - type: command
          command: ".claude/hooks/nome/validate.sh"
  PostToolUse:                   # depois da tool executar
    - matcher: "Edit|Write"
      hooks:
        - type: command
          command: ".claude/hooks/nome/run-tests.sh"
  Stop:                          # ao encerrar (vira SubagentStop)
    - hooks:
        - type: command
          command: ".claude/hooks/nome/verify.sh"
---
```

Pontos de atenção: hooks do frontmatter disparam quando o agent roda como **sub-agent** (Task tool ou @mention) — não quando é a sessão principal. Stop hooks do frontmatter são convertidos automaticamente em `SubagentStop`: rodam antes de retornar o resultado ao agent principal. Além do `type: command`, existem handlers `prompt` (avaliação semântica por um modelo) e `agent` (spawna um sub-agent verificador com Read/Grep/Glob).

3 Padrões por tipo de agent

Agent somente leitura (reviewer, auditor, explorador)

```
hooks:
  PreToolUse:
    - matcher: "Write|Edit|Bash"
      hooks:
        - type: command
          command: |
            echo 'Este agent é somente leitura.' >&2
            exit 2   # exit 2 = bloqueia a ação
```

Agent de implementação (backend, frontend, parser)

```
hooks:
  PostToolUse:
    - matcher: "Edit|Write"
      hooks:
        - type: command
          command: ".claude/hooks/backend/run-module-tests.sh"
  PreToolUse:
    - matcher: "Bash"
      hooks:
        - type: command
          command: ".claude/hooks/backend/validate-scope.sh"
  Stop:
    - hooks:
        - type: command
          command: ".claude/hooks/backend/verify-before-return.sh"
```

Agent de QA: só comandos de teste passam

```
#!/bin/bash
# .claude/hooks/qa/allow-only-tests.sh
INPUT=$(cat)
CMD=$(echo "$INPUT" | jq -r '.tool_input.command // empty')

ALLOWED="^(pytest|npx vitest|npx jest|npm test|python -m pytest)"

if ! echo "$CMD" | grep -qE "$ALLOWED"; then
  echo "QA Agent: apenas comandos de teste. Recebido: $CMD" >&2
  exit 2
fi
exit 0
```

4 OpenCode: hooks por agent via plugin

O OpenCode não tem frontmatter de hooks por agent — o lugar certo é um plugin TypeScript que verifica qual agent está agindo:

```
// .opencode/plugin/agent-hooks.ts
import type { Plugin } from "@opencode-ai/plugin"

export const AgentHooks: Plugin = async ({ client, $ }) => {
  return {
    "tool.execute.before": async (input, output) => {
      const agent = (input as any).agentName

      // code-reviewer: somente leitura
      if (agent === "code-reviewer" &&
          ["write", "edit", "bash"].includes(input.tool)) {
```

```
        throw new Error("code-reviewer é somente leitura")
      }

      // qa-agent: só testes
      if (agent === "qa-agent" && input.tool === "bash") {
        const cmd = (input.args as any)?.command || ""
        if (!/^(pytest|vitest|jest|npm test)/.test(cmd)) {
          throw new Error(`qa-agent: comando não permitido: ${cmd}`)
        }
      }
    },

    "tool.execute.after": async (input) => {
      const agent = (input as any).agentName
      // backend: testes após edição
      if (agent === "backend-agent" &&
          ["edit", "write"].includes(input.tool)) {
        const r = await $`pytest tests/ -q --tb=line`.quiet().nothrow()
        if (r.exitCode !== 0) {
          await client.session.prompt({
            text: `Testes falhando:\n${r.stderr}`, noReply: false
          })
        }
      }
    }
  }
}
```

Mapa de portabilidade: `PreToolUse` → `tool.execute.before` · `PostToolUse` → `tool.execute.after` · `Stop/SessionEnd` → evento `session.idle`.

5 Organização dos scripts

Um diretório por agent — manutenção e review ficam triviais:

```
.claude/hooks/
├── backend/
│   ├── validate-scope.sh          ← PreToolUse Bash
│   ├── run-module-tests.sh        ← PostToolUse Edit|Write
│   └── verify-before-return.sh    ← Stop
├── parser/
│   ├── validate-output.sh         ← PostToolUse Write
│   └── check-encoding.sh          ← Stop
├── qa/
│   └── allow-only-tests.sh        ← PreToolUse Bash
└── reviewer/
    └── block-writes.sh            ← PreToolUse Write|Edit|Bash
```

6 Prompt pronto: gerando os hooks dos agents

▸ PROMPT PARA CRIAR OS HOOKS

```
Leia os agents em .claude/agents/ e o PLAN.md na raiz.

Para cada agent, crie os hooks do seu domínio e escopo:

AGENT SOMENTE LEITURA (reviewer, auditor):
- PreToolUse bloqueando Write, Edit e Bash,
  com mensagem clara do porquê

AGENT DE IMPLEMENTAÇÃO (backend, frontend, parser):
- PostToolUse rodando os testes do módulo após edição
- PreToolUse bloqueando comandos fora do escopo
- Stop verificando testes antes de encerrar

AGENT DE QA:
- PreToolUse permitindo APENAS comandos de teste

Claude Code: seção hooks no frontmatter de cada agent,
scripts em .claude/hooks/nome-do-agent/,
exit 2 bloqueia / exit 0 permite.

OpenCode: .opencode/plugin/agent-hooks.ts verificando
o agentName em tool.execute.before / tool.execute.after.

Ao terminar, demonstre um caso concreto: um agent tentando
algo fora do escopo e sendo bloqueado com mensagem clara.
```

7 Erros mais comuns

ERRO	SINTOMA E CORREÇÃO
Hook do frontmatter "não dispara"	O agent está rodando como sessão principal — frontmatter só vale para sub-agent (@mention/Task tool).
Script sem `exit 0` no final	O shell retorna o exit code do último comando — que pode ser ≠ 0 sem erro real, bloqueando sem querer.
Loop infinito no Stop hook	Verifique `stop_hook_active` no início. Sem isso: hook bloqueia → agent tenta encerrar → hook dispara → ciclo.
Scripts sem permissão de execução	`chmod +x .claude/hooks/**/*.sh` — sem isso os hooks falham em silêncio.
Procurar frontmatter de hooks no OpenCode	Não existe. O plugin TypeScript é o mecanismo — e a lógica "qual agent" é sua, via contexto do evento.

▸ COLA PARA FIXAR

- System prompt **pede**; hook **garante**. Guardrail de verdade é determinístico.
- Globais protegem o projeto; por agent protegem o **contrato do domínio** — use os dois.
- No Claude Code, hooks globais **não cobrem sub-agents**: o frontmatter é obrigatório para delegação segura.
- Padrões: leitura-apenas **bloqueia Write/Edit/Bash**; implementação **testa após editar**; QA **só roda testes**.
- `exit 2` bloqueia, `exit 0` permite; Stop hooks precisam checar `stop_hook_active`.
- OpenCode: plugin TS com `tool.execute.before/after` fazendo o papel do frontmatter.